

Tic Tac Toe

Course: CSD-311 Artificial Intelligence

Team Details

Name	Roll Number
Diya Dugar	2210110831
Jahanvi Singh	2210110914
Nandini Kataria	2210110797
Pratyush Kamal	2210110475
Rishika	2210110509
Urvi Gupta	2210110950

Abstract

This project focuses on developing an intelligent Tic Tac Toe game with multiple difficulty levels, implemented using **Pygame**, a Python library for game development. The game offers three levels of difficulty: **Easy**, where the AI makes arbitrary moves; **Medium**, where the AI strategically blocks the opponent's winning moves; and **Hard**, which utilises the Minimax Algorithm and its optimised version with Alpha-Beta Pruning. The Minimax Algorithm ensures optimal gameplay by evaluating all possible moves, while Alpha-Beta Pruning enhances efficiency by reducing the number of evaluated game states. By combining a user-friendly interface with robust AI techniques, this project provides an engaging and interactive experience for players of all skill levels.

Introduction

Background

Tic Tac Toe is a classic game of strategy played on a 3x3 grid. The first traces of tic-tac-toe go back to Egypt, with remnants of 3x3 game boards on roofing tiles from 1300 BCE. The game

provides an excellent platform for exploring concepts in artificial intelligence, such as decision-making and game theory. AI plays the role of one of the players, competing against the human user with varying levels of strategic sophistication.

Task Environment

The following defines the task environment of the game :

1. **Deterministic:** Outcomes are pre-determined by player moves.
2. **Sequential:** Each move directly impacts the game's progression.
3. **Static:** The environment remains unchanged unless a move is made.
4. **Discrete:** The 3x3 grid offers a finite number of defined states.
5. **Fully Observable:** Both players can view the complete state of the game board.
6. **Multiple Agents:** The AI competes against the human player.

Proposed Model

The Tic Tac Toe game is structured into three difficulty levels:

1. Easy: Random moves without strategy.
2. Medium: Basic two-step strategic thinking.
3. Hard: Optimal decision-making using the Minimax Algorithm and Alpha Beta Pruning.

Methodology

Easy Level Implementation

AI Behavior: Selects an arbitrary empty cell for its move.

Logic: Does not involve any foresight or strategy.

Medium Level Implementation

AI Behavior: Introduces blocking strategy.

Logic: checks for moves that make the opponent win. Blocks the move. It places its piece at an arbitrary position if no such move exists.

Hard Level Implementation

AI Behavior: Uses the minimax algorithm with alpha-beta pruning.

Logic: imitates all possible outcomes of the game and then chooses a branch which maximises its winning chance

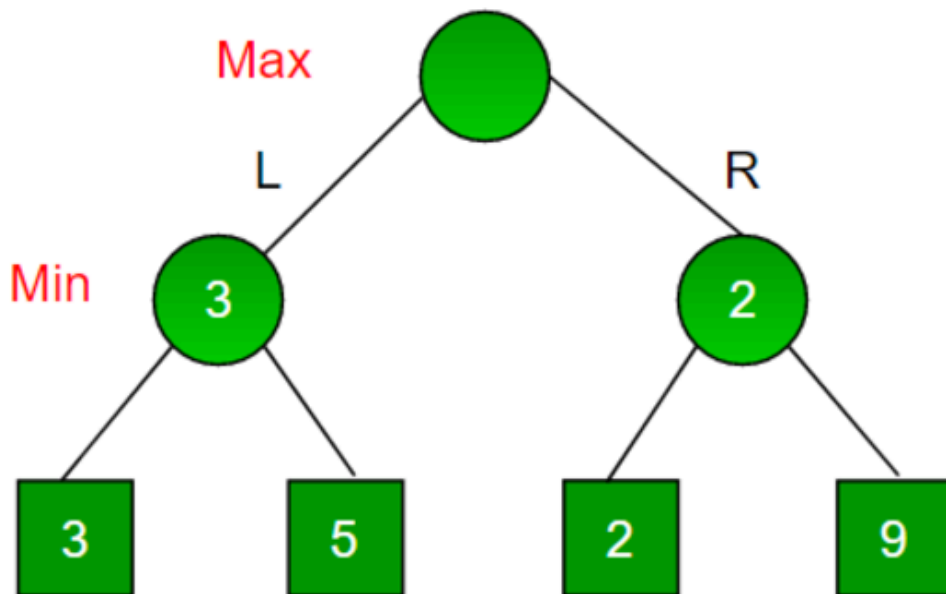
Algorithms Used

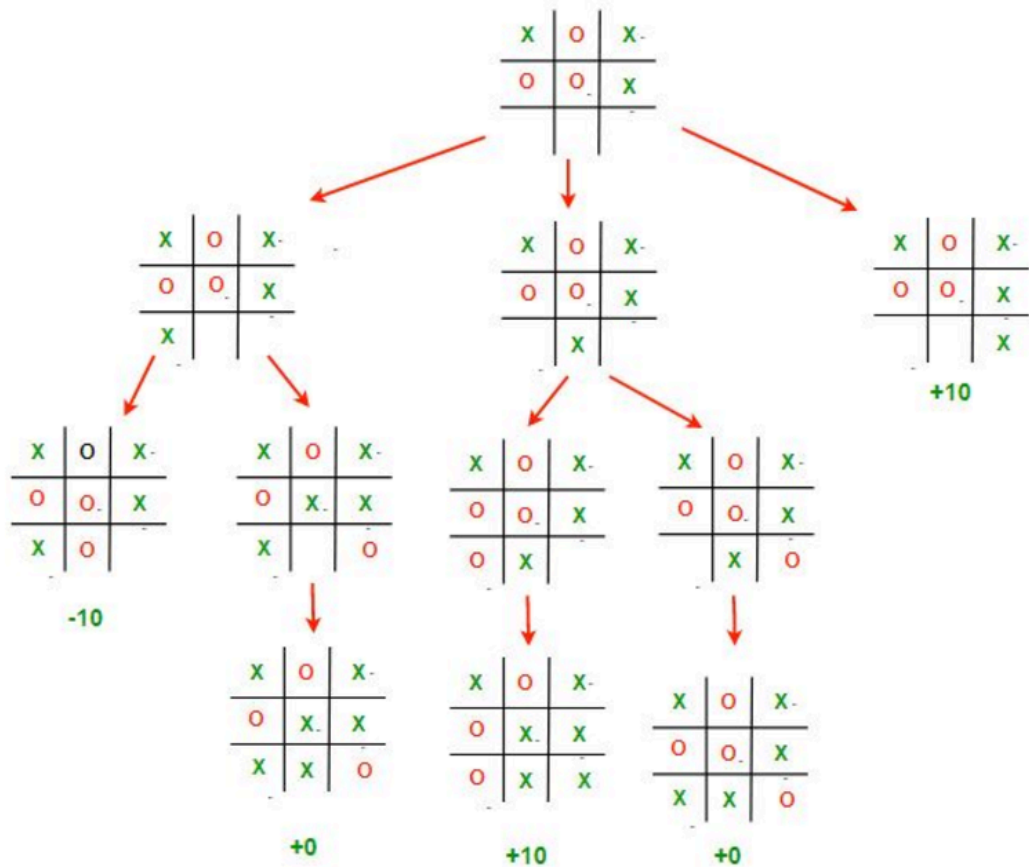
The Minimax Algorithm

Minimax is a backtracking algorithm is used in decision-making and game theory to find the optimal move for a player, assuming that your opponent also plays optimally. It is widely used in two-player turn-based games such as Tic-Tac-Toe, Backgammon, Mancala, Chess, etc.

In Minimax, the two players are called maximiser and minimiser. The maximiser tries to get the highest score possible, while the minimiser attempts to do the opposite and get the lowest score possible.

Every board state has a value associated with it. In a given state, if the maximiser has the upper hand, the board's score will have some positive value. If the minimiser has the upper hand in that board state, it will tend to have some negative value. The board's values are calculated using unique heuristics for every type of game.





Heuristics used for tic tac toe :

- **+10** for the AI player winning
- **0** for a tie
- **-10** for the opponent winning

Time complexity:

- $O(b^d)$ b is the branching factor, and d is the count of depth or ply of a graph or tree.

Space Complexity

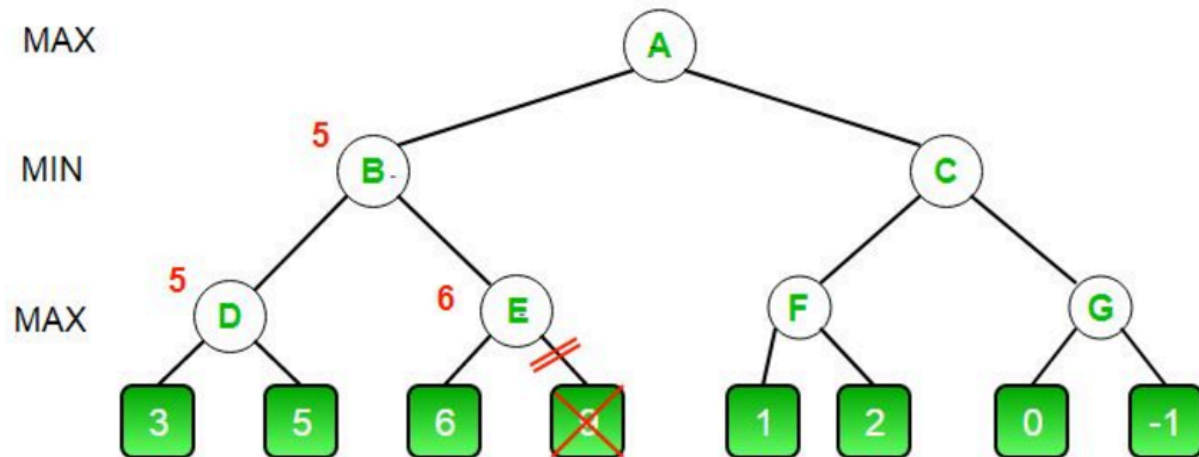
- $O(bd)$, where b is the branching factor and d is the maximum depth of the tree, is similar to DFS.

The Minimax Algorithm with Alpha Beta Pruning

Alpha-Beta pruning is not a new algorithm but an optimisation technique for the minimax algorithm. It reduces the computation time considerably. This allows us to search much faster and even deeper levels in the game tree. It cuts off branches in the game tree that need not be searched because a better move already exists. Alpha-Beta pruning is called because it passes two extra parameters in the minimax function: alpha and beta.

The parameters are defined as alpha-beta.

Alpha is the best value the maximiser can currently guarantee at that level or above.
Beta is the best value the minimiser can currently guarantee at that level or below.



Time complexity:

- $O(b^{d/2})$ b is the branching factor, and d is the average case's count of depth or ply of graph or tree.
- $O(b^d)$ for worst case.

Space Complexity

- $O(bd)$ where b is the branching factor into d is the maximum depth of the tree similar to DFS.

Experiment

We conducted an experiment and played ten games of each level, and the results were as follows.

The winning represents the AI against an experienced player.

Levels	Games Won	Games Tied	Games Lost
Easy	0	0	10
Medium	1	3	6
Hard	2	8	0

Time Analysis for playing the first move

Easy level

- 0 seconds

Medium Level

- 0 seconds

Hard Level (without alpha-beta pruning)

```
Time taken for the AI to play the first move: 0.049406 seconds
```

```
Time taken for the AI to play the first move: 0.066607 seconds
```

```
Time taken for the AI to play the first move: 0.051574 seconds
```

```
Time taken for the AI to play the first move: 0.056890 seconds
```

```
Time taken for the AI to play the first move: 0.063654 seconds
```

Average time taken: 0.0576262 seconds.

Hard Level (with alpha-beta pruning)

```
Time taken for the computer to play the first move: 0.029246 seconds
```

```
Time taken for the computer to play the first move: 0.020991 seconds
```

```
Time taken for the computer to play the first move: 0.021991 seconds
```

```
Time taken for the computer to play the first move: 0.020990 seconds
```

```
Time taken for the computer to play the first move: 0.021996 seconds
```

Average time taken: 0.0230428 seconds.

- The overall performance of the code significantly improved by using alpha-beta pruning.

Conclusions

The development of this Tic Tac Toe game demonstrates the effective use of game AI algorithms across multiple difficulty levels. By implementing **Easy** and **Medium** modes, the game provides an approachable experience for casual players, with the Easy level making random moves and the Medium level strategically blocking winning moves. In contrast, the **Hard** level, powered by the **Minimax Algorithm** and optimised with **Alpha-Beta Pruning**, guarantees unbeatable performance, ensuring that the AI always secures a win or a draw when played against an optimal opponent. The Minimax Algorithm evaluates all possible game states to make optimal decisions, while Alpha-Beta Pruning enhances efficiency by pruning irrelevant branches of the decision tree.

Built using **Pygame**, this project offers a dynamic and engaging user experience and highlights the practical application of AI techniques in game design. The hard level's unbeatable nature underscores these algorithms' effectiveness, demonstrating how logic and optimisation can create an AI capable of simulating perfect play.

Limitations

- Our analysis was limited to a 3X3 grid as tic tac toe is a game played on a board of that size.
- Heavy computation time as it has an exponential time complexity for the hard level.

References

1. Russell, S., & Norvig, P. (2020). Artificial Intelligence: A Modern Approach.
2. Stack Overflow. "Optimizing Game AI Using Minimax." <https://stackoverflow.com>
3. <https://momath.org/wp-content/uploads/2021/08/Alyssa-Choi-Tic-Tac-Toe.pdf>
4. <https://www.geeksforgeeks.org/minimax-algorithm-in-game-theory-set-1-introduction/>